

-



- [Core Concepts](#)
- Data Sources

On this page

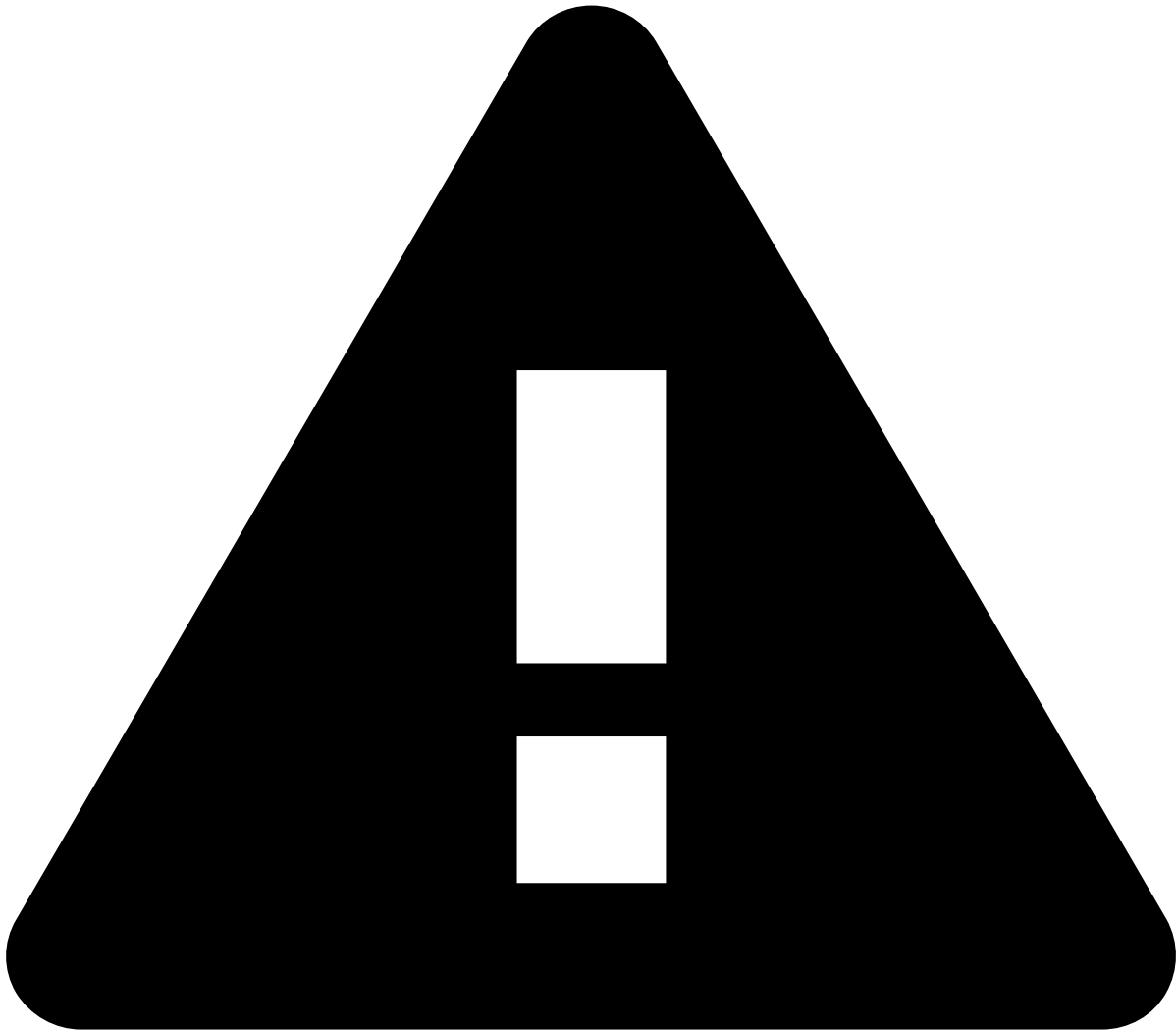
Data Sources

Was this page helpful? 

Data sources represent datasets that will be used for data science purposes within Pulse. An example of a source of data is a CSV file.

All data science concepts of the [data science loop](#) depend on data or its schema:

- [Models](#) and [rules](#) use data sources to perform decisions.
- [Plans](#) receive a data source as input, and generate a transformed data source as output.



warning

When you create a new data source, you never load the actual data into the system. Pulse only maintains a pointer to the physical location of the data file, as loading the whole data set would be too memory-consuming. Pulse fetches the data from the file(s) from the physical location whenever a data science task requires it.

Do not delete the data file from your system, otherwise any subsequent data science tasks that depend on it will not work.

Data fields📄📄

Data sources have **fields**. For example, in a CSV file, these correspond to the columns. Each field is characterized by a [data type](#), which defines the possible values of the field.

A data source is comprised of records. For example, in a CSV file, each record is represented as a row. Typically, each record contains a value for each field. However, it is possible for a field in a record to have no value, i.e. *missing value*.

The following image shows how Pulse displays fields and values of a data source.

Data schemas📄📄

When you create a data source and import information into Pulse, it also creates a **schema**. A schema describes the available data fields of a data source, as well as their [types](#).

Data might come from different sources and in different ways, and be manipulated at almost every stage of your project. Schemas, therefore, provide formatting and structure for data source manipulation/exploration during a data science stage and for real-time data coming into Pulse in Production.



info

All intermediate data manipulations in Pulse, whether during the Data Science stage or in Production, generate new schema instances. However, these schemas are internal and not accessible to characterize runtime data.

To translate the underlying (and implicitly created) data schema of a data source into a schema that you can configure for a runtime [workflow](#), you must explicitly export the data source to a schema.

As an accelerator, the workflow allows you to directly copy the schema from an existing data source.

Schema immutability

When you successfully create a data source and import data into Pulse, it automatically also defines the corresponding schema. Once you have a data source, you cannot edit or update its data schema.

Data schemas, and consequently, data sources are immutable due to the fact that most data science concepts and tasks fully rely on data sources. Therefore, to keep your project consistent, whenever you change the underlying structure of a data source, Pulse would have to propagate those changes further down your data science pipeline to [plans](#), [models](#), and [rules](#), where the data source has been used. Additionally, this would have potentially disastrous and hard-to-solve implications if you were already using plans, models, and rules in a runtime [workflow](#).

However, changing a data schema after it has been involved, for example, in training a model, wouldn't even make sense, since model configurations and results are, by definition, also immutable.

On the other hand, changing a data source schema, with different field names and data types, and not propagating changes would, for example, break plans that were already configured to consume data in a specific way.



info

If you need to change the data schema of your data sources, you can do it by [duplicating the data source and then editing the data configuration](#).

In reality, since the schema is immutable, this effectively generates a new data source with an updated schema.

In a data science process, whenever your data moves along a plan and suffers transformations, the data schema of the input data source remains unchanged. To reflect the result of data manipulation and transformations, since schemas are immutable, Pulse generates a new data source with a new schema. The subsequent stages in your data science pipeline should be configured accordingly to use the output data source.



info

Since everything relies on data, every time you use data and its schema definition during the data science stage, Pulse must ensure that everything is correctly translated into the runtime environment.

Every time you create/configure an entity that requires data and can be deployed directly into Production, Pulse uses a copy of the original schema to maintain the integrity of the original. For example, when specifying the input data schema for a [deployable plan](#) or when configuring a [rules snapshot](#).

Compatible data sources ⓘ ⓘ

In data science mode, operations are always done over the data source files that you import, such as CSV or Parquet files. After deployment, the data can come in different formats. For example, you can receive data over a socket reading

ISO8583 messages or via a REST API. Production data must be compatible with your data sources to ensure the data processing operations will continue to work.

Moreover, if you want to, for example, run an evaluation on a trained model using different data, you also need to make sure it is compatible with the dataset originally used to train the model. Otherwise, the experiment will not be useful for your data science project.

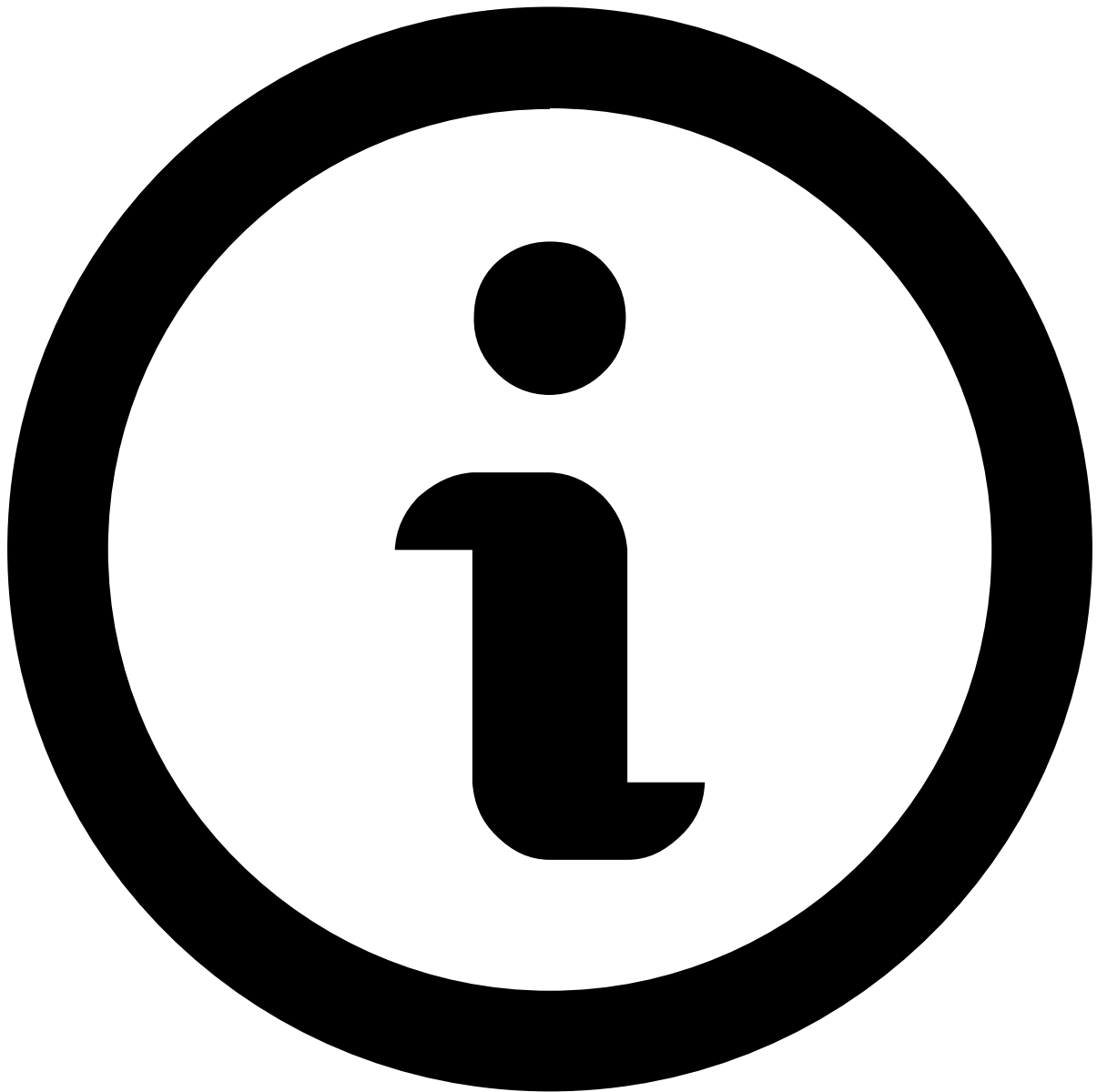
In Pulse, two data sources are considered compatible according to the following table:

Concept	Compatibility strategy	Description
Plans	Equivalent	New schema must be equal to the one of the original data. Data must have the exact same format (type and possible values), order and names.
Models	Superset	New schema can have any fields as long as it contains all fields of the original schema, with the exact same format (type and possible values) and names. The order is irrelevant in this case.
Rules	Superset	New schema can have any fields as long as it contains all fields of the original schema, with the exact same format (type and possible values) and names. The order is irrelevant in this case.

Invalid instances🔗📄

Invalid instances are data rows (aka. instances, records, or events) that could not be read from the dataset for one of the following reasons:

- The data row contains more or less data fields than the schema.
- The data in one or more fields of the row is not in the format that the schema expects. This has three cases:
 - The type of the input data does not match the data type defined in the schema. Example: "10.0", when the field in the schema is an int, instead of a double.
 - The format of the input data does not match the format configurations of the data type defined in the schema. Example: "01-01-1990", when the field in the schema is a Date Time configured with the YYYY-MM-DD format.
 - The data field is a categorical, and the value of the input data is not in the list of the possible values defined in the schema. Example: "CAN", when the categorical type of the schema is configured to accept "CAD", "CNY", "EUR" and "USD".



note

Invalid instances are ignored in data science activities, for example when calculating metrics in a [data source analysis](#), executing a [Plan, training a model](#), etc.

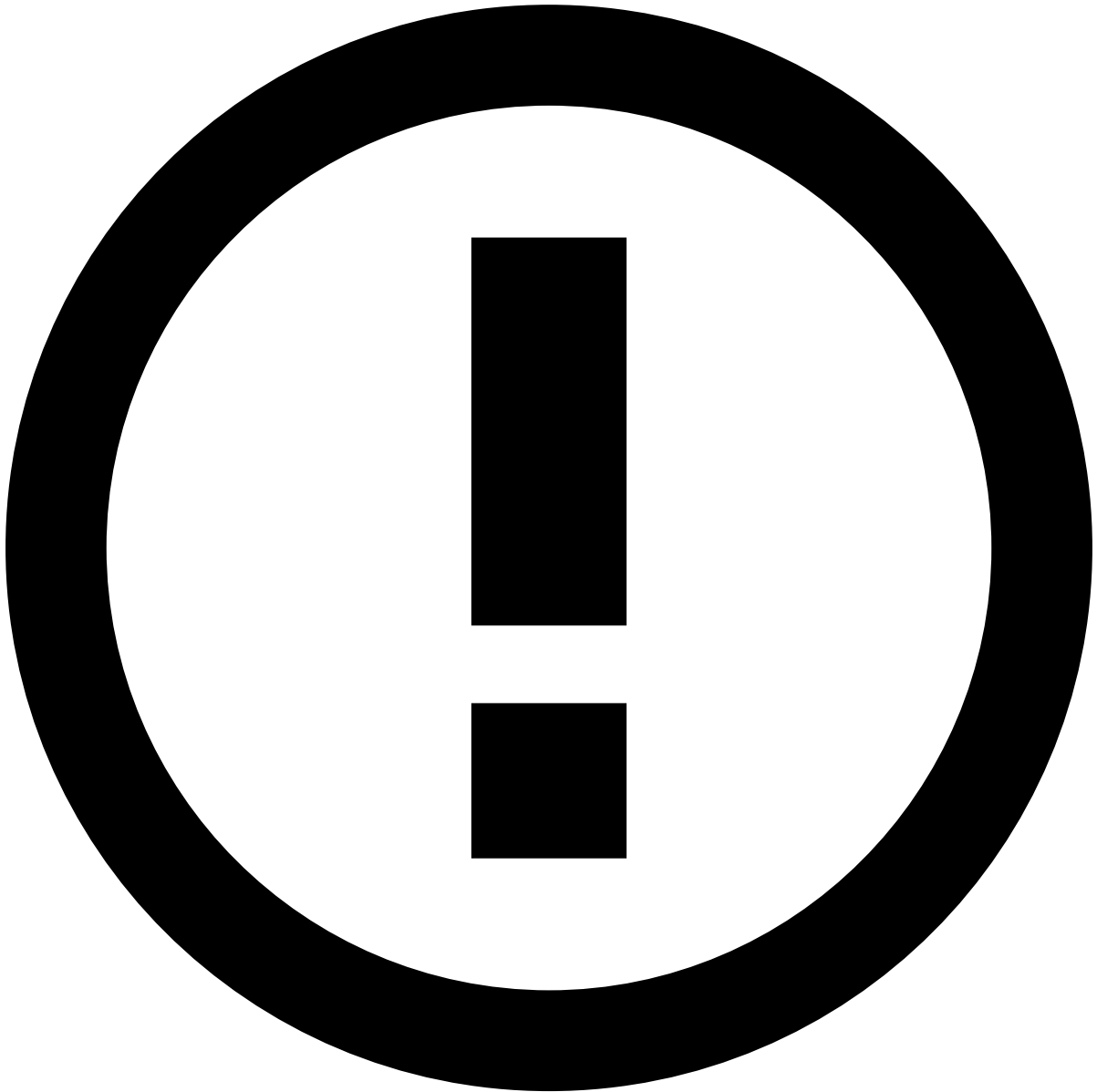
The Pulse UI shows information about invalid instances after you run a [data source analysis](#).

To fix invalid instances, after investigating their cause in the UI, you can combine the following two approaches:

- Change the data.
Preprocess the dataset externally (e. g. using Python), to transform invalid values into valid ones.
This approach helps when in the same data field, in different records there are different types or formats.
- Change the schema.
Duplicate the data source, and in the new data import, change the data type or the configuration of the data type.
This can help if there is an issue with the field's data type in the schema, affecting all values of that field. For instance, in the case of a missing categorical value, or if you picked the wrong setting when defining a data type.

Missing values

Some records in your dataset will most likely have empty fields (perhaps caused by faulty equipment). These, if untreated, would also lead to misleading results or failure during the training. Hence Pulse allows you to define how to represent missing values.



info

When you create a data source from a CSV file, you can define any of the following representations to be treated as missing values:

- NULL
- null
- N/A
- Empty string

When you run an [analysis on your data source](#), it might indicate that the count of missing values is very high for an important field. In these cases, you may need to re-engineer your features or build a [data cleaning plan](#) that detects and removes records that match the missing value representation you configured.

In case there are missing values for a target variable, those records are ignored in [model evaluations](#) and [rule evaluations](#).